

Variables en python

Cuando nosotros creamos una variable y le asignamos un valor, estamos reservando un espacio de memoria en la ram, entonces la direccion de memoria estara en Hexadecimal y empezara 0x333, "0x" nos dice q esta en hexadecimal. Cuando cambiamos el valor de la variable no cambia la direccion de memoria solo se sobrescribe del valor anterior y sigue teniendo la misma direccion de memoria.

Seccion 3

Variables

1. Al crear una variable esta en memoria y el valor es un objeto, entonces cuando sobrescribe no es que borra el nuevo valor sino que se crea un nuevo objeto y ahora apunta a ese nuevo valor.
2. si quieres que un valor en python sea un numero que no lo vas a usar para sumar,etc. Sino que funcione como string lo pones entre comillas.

Convecciones y buenas practicas para las variables

1. Snake case: Que significa que los nombres esten en minuscula y que esten separados por un guion bajo y evitar empezar con digitos.
2. Los nombres deben ser descriptivos no usar solo una letra sino un nombre
3. Evitar nombres de un solo caracter.

Tipos de datos

Python es dinamico por lo que no necesitamos indicar que tipo de variables son.

1. Numero int: Numeros enteros.
2. Numero con punto flotante(float): Numeros con coma.
3. Cadena de texto: Cadena de letras o caracteres.
4. Booleanos: Almacenan un valor logico, verdadero o falso, y lo usaremos para controlar el flujo del programa
5. none: este es un tipo especial de python que representa ausencia de valor.

Constantes

Python es diferente que otros lenguajes de programacion debido a que no tiene una especificacion para definir una constante, por lo que para una buena practica de programacion vamos a usar una convencion que es poner la variable en cuestion en mayusculas y con esto entender que esa variable no debe ser modificada.

Seccion 4

Cadenas

La cadena o string es el tipo de dato que se usa para almacenar una secuencia de caracteres, se cierran en comillas dobles o simples. Los caracteres en cuestion pueden ser tambien numeros o espacios.

Ejemplo:

```
#Numero de cadenas
cadena1 = "Hola mundo"

#Cadena con numeros
cadena 2 = "123 456"
```

Detalle de una cadena

Los caracteres de una cadena estan indexados de manera secuencial. Por lo tanto, podemos acceder cada caracter indicando el indice del caracter que queremos recuperar.

Ejemplo:

0	1	2	3	4	5	6	7	8	9
"h"	"o"	"l"	"a"	" "	"m"	"u"	"n"	"d"	"o"

Para calcular el indice es el numero de caracteres y espacio(n) menos 1: $n-1$

Immutabilidad de una cadena

Al crear una nueva cadena los caracteres dentro de ella no pueden ser modificados. Si queremos modificar una cadena, entonces tendremos que crear una nueva cadena.

Caracteres Especiales

Se pueden incluir caracteres especiales, como una diagonal invertida `"\"`.

- Nueva linea: `"\n"` Inserta un salto de linea.
- Tabulacion: `"\t"` Inserta un tabulador horizontal, para alinear texto.
- Comilla simple: `"'"` Permite incluir comillas simples en una cadena delimitada por comillas simples.
- comilla doble: `"\""` Permite incluir comillas dobles en una cadena delimitada por comillas dobles.
- Barra invertida: `"\""` Permite incluir una barra invertida en la cadena.

Existen mas pero estos son los esenciales.

Concatenacion de cadenas

Es una operacion que permite combinar dos o mas cadenas para formar una cadena nueva, en python hay varias formas.

- Uso de operador `"+"`: Es el mas directo para concatenar cadenas.
- Uso de la funcion `join`: Nos permite unir cuantas cadenas nosotros necesitemos. ej:

```
" ".join(["cadena1", "cadena2", "cadena3"]) #"" o " " el espacio entre cadenas.
```

Formateo de Cadenas

Python ofrece varias formas de formatear cadenas, que incluyen la capacidad de concatenar texto, variables e incluso dar otro tipo de formateo, como por ejemplo indicar el numero de decimales a utilizar en el formato.

- f-string(python3.6+): esta es la opcion mas recomendada, por ser la mas sencilla, rapida y legible.

```
resultado = f' Hola {variable}.'
```

- Metodo format: Es muy versatil y podertoso. Permite construir cadenas muy complejas.

```
resultado = 'Hola {}'.format(variable)
```

Metodos de Cadenas

Las cadenas en python vienen con una serie de metodos utiles que facilitan su manipulacion.

- upper(): Cambia las letras a mayusculas.
- lower(): Cambia las letras a minusculas.
- strip(): Elimina los espacios, al inicio y al final de una cadena.

Obtener el largo de una cadena

Es una funcion que contiene python, llamada len(). La funcion len funciona para varios tipos de datos.Cuando se calcula el largo de una cadena se incluye o se toma en cuenta todos los caracteres de una cadena hasta los espacion en blanco, caracteres eseciales, etc.

```
cadena1 = 'Hola, mundo!'
longitud = len(cadena1) #Devuelve largo de 12
```

Subcadenas en python

La subcadena es una parte de una cadena principal, y hay varias maneras de extraer subcadenas en python. Podemos extraer, reemplazar, entre otras operaciones.

- Extraccion de cadenas:(Slicing): El slicing o segmentacion permite indicar el indice del inicio y el inidce final(sin incluir este ultimo caracter)

```
subcadena = cadena [inicio:fin]
```

- Buscar subcadenas (find): El metodo devuelve el indice de la primera aparicion de la subcadena, pero si no encuentra la subcadena, devuelve -1.

```
cadenas = 'Hola Mundo'
posicion = cadena.find("Mundo")
print(posicion) #imprime 5
```

- Reemplazar subcadenas(replace): El metodo reemplaza una subcadena por otra dentro de una cadena principal

```
cadenas = 'Hola Mundo'
nueva_cadena = cadena.replace('Mundo','a todos')
print(nueva_cadena) # 'Hola a todos'
```

- Extraer subcadenas por separadores(split): La funcion split permite dividir una cadena en una lista de subcadenas basadas en un caracter separador.

```
datos = 'Juan, 30, Mexico'
lista = datos.split(',')
print(lista) # ['Juan','30','Mexico']
```

Seccion 5

Conversion de tipo de datos

Otra forma de llamarlo es casting, es una tecnica para manipular datos que no estan en el tipo requerido. Podemos hacer conversiones desde y hacia tipos de datos.

- Convertir a entero: funcion int()
- Convertir a flotante: funcion float()
- Convertir a cadena: funcion str()
- Convertir a booleano: funcion bool()

Entrada de datos por consola

Entrada de datos se realiza usando la funcion input. Esta funcion pausa el programa y espera a que el usuario ponga algun texto. Una vez que el usuario ponga enter, el texto introducido se vuelve como una cadena (str).

Caracteristicas de la funcion input

- Interactividad: Permite a los usuarios de nuestro programa proporcionar valores dinamicos, en lugar de usar valores estaticos.
- sencillez: Es sencilla de usar y solo necesita indicar la cadena o mensaje a mostrar al usuario, para que entienda lo que se esta solicitando.
- Tipo de dato: Siempre devuelve una cadena, si requiere otro tipo de dato hay q convertirlo.

Generar valores aleatorios

La funcion randint(), que es parte del modulo random, nos permite generar numeros aleatorios. randint(a,b), devuelve los valores entre a y b, incluyendo los valores a y b. Es importante importarr el modulo random antes de usar la funcion. Para importar el modulo usamos:

```
import random
```

Seccion 6

Operadores en python

Son simbolos especiales que estan diseñados para hacer una operacion en especifico. Tenemos varios tipos:

- Operadores aritmeticos: Permiten realizar calculos matematicos basicos.
- Operadores de asignacion: Se utilizan para asignar valores a variables.
- Operadores de comparacion: Se utilizan para comparar un valor con otro.
- Operadores Logicos: Se utiliza para expresiones condicionales o logicos.
- Operadores con identidad: Se utilizan para comparar si dos variables son el mismo objeto.
- Operadores de membresia: Se presenta para probar si una secuencia(ej:subcadena) se presenta en un objeto.

Operadores aritmeticos

Para realizar calculos matematicos:

- suma(+)
- resta(-)
- multiplicacion(*)
- division(/)
- division entera(//): Divide el primer operando con el segundo. El resultado te dara solo la parte entera.
- modulo(%): Residuo d ela division
- exponente(**): eleva el primer operador a la segund potencia.

Operadores de asignacion

El operador de asignación se usa para asignar un valor a una variable, y se usa el caracter (=) para ello.

Ejemplo:

```
variable = valor  
numero = 10
```

Python tambien tiene lo que es la asignacion multiple lo que permite asignar valores a varias variables en una misma linea de codigo.

Ejemplo:

```
# Sintaxis de asignacion multiple
variable 1, variable 2 = valor 1, valor 2
a,b,c = 1,"carro", 5
```

Tambien hay asignacion en cadenado en python. Esto permite asgnar el mismo valor a diferentes variables en una linea de codigo.

Ejemplo:

```
# Sintaxis de asignacion encadenada
variable1 + variable 2 + .... =valor
# Ejemplo de inicializar contadores
contador1 + contador2 = 0
```

Otra caracteristica es que podemos pedirle al usuario multiples valores de entrada, usamos la funcion split(), para que el programa entienda que un espacio es el separador de una a otra variable.

Ejemplo:

```
# Sintaxis de pedir multiples valores de entrada, si dejmaos el split entiende
que es un espacio
nombre, apellido = input('Ingrese su nombre y apellido: ').split()
# Tambien podemos poner dentro de la funcion split cual queremos q sea nuestro
separador
nombre, apellido = input('Ingrese su nombre y apellido: ').split(",")
```

Operadores de asignacion compuesto

Los operadores de asignacion compuesto cambian una operacion aritmetica con una asignacion, haciendo las operaciones mas concisas. Los operadores pueden ser +=, -=, *=, /=, etc.

Ejemplo:

```
# Sintaxis operador asignacion compuesto
variable Operador= Valor
# Ejemplo
contador = 0
contador += 1 # contador = contador + 1
```

Operador de Comparacion

Los operadores de comparacion se utiliza para comparar dos valores. El valor siempre es un valor booleano 'true' o 'false', dependiendo si la condicion se cumple o no.

- Operador de igualdad

```
# Sintaxis operador de igualdad ==  
a == b  
# Ejemplo  
print(5 == 5) #Imprime true  
print(5 == 3) #Imprime false
```

- Operador de desigualdad

```
# Sintaxis operador de igualdad !=  
a != b  
# Ejemplo  
print(5 != 6) #Imprime true  
print(5 != 5) #Imprime false
```

- Operador menor que(<)

```
print(5 < 6) #Imprime true  
print(5 < 5) #Imprime false
```

- Operador menor o igual que(<=)

```
print(5 <= 6) #Imprime true  
print(5 <= 4) #Imprime false
```

- Operador mayor que(>)

```
print(5 > 3) #Imprime true  
print(5 > 7) #Imprime false
```

- Operador mayor o igual que(>=)

```
print(5 >= 3) #Imprime true  
print(5 >= 7) #Imprime false
```

Operadores logicos

Se utilizan para realizar operaciones logicos

- Operador Logico and (y): Devuelve True si ambos operadores son verdaderos, caso contrario siempre sera falso.

```
exp1 = False
exp2 = True
print(exp1 and exp2) #False
```

- Operador logico or (o): Devuelve True si cualquiera de los operandos son verdaderos, sino tiene ninguna variable verdadero siempre sera falso.

```
exp1 = False
exp2 = True
print(exp1 or exp2) #True
```

- Operador logico not (no): Inverte el valor del operado. Es un operador unario.

```
exp1 = False
print(not exp1) #True
```

Precedencia de Operadores

Determina el orden en que se evaluan las operaciones.

Python aplica las siguiente tabla para asegurar que algunos operadores tengan prioridad sobre otros

1. Operador de parentesis ()
2. Exponente **
3. Unarios +x(positivo), -x(negativo)
4. Multiplicacion, division, modulo *, /, //, %
5. Suma y Resta +, -
6. Comparacion ==, !=, <, <=, >, >=
7. Operadores logicos not, and y or
8. Operadores de asignacion =, +=, -=, /=, %=, //=, **=

```
resultado = 5 + 3 * 2 ** 2 # 17
resultado = (5 + 3) * 2 ** 2 # 32
```

Seccion 7

Sentencias de decisión

Nos permite controlar el flujo de ejecucion del programa

Las estructuras pueden ser: if, else y elif

- La sentencia if permite ejecutar un bloque de código si la condición a evaluar es verdadera.

Ejemplo:

```
# Sintaxis sentencia if
if condition:
    #Bloque de código que se
    #ejecuta si la condición es TRUE
    #son importantes la tabulación

# Ejemplo
edad = 30
if edad >= 30:
    print("Eres mayor de edad")
```

Diagrama de Flujo

Un diagrama de flujo es una representación gráfica de los pasos a ejecutar para lograr un resultado específico.

Se utilizan símbolos estandarizados para representar distintos tipos de acciones.

1. Círculo o Ovalo: Representa el inicio o fin de un proceso
2. Rectángulo: Muestra instrucciones o acciones a ejecutar
3. Rombo o diamante: Indica decisiones, con múltiples flujos dependiendo si la respuesta es verdadera o falsa
4. Flechas: Dirigen el flujo del proceso, mostrando la dirección en que se mueven la secuencia de acciones

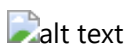
Ejercicio:

Dado el siguiente código, vamos a crear su diagrama de flujo equivalente:

- Código

```
edad = 30
if edad >= 30:
    print("Eres mayor de edad")
```

- Diagrama



Sentencia else

La sentencia else se usa para ejecutar un bloque de código cuando la sentencia if es falsa

```
# Sintaxis sentencia else-if
if condition:
    #Bloque de codigo
    #si la condicion es verdadera
else:
    #Bloque de codigo
    #si la condicion es falsa
#Ejemplo
edad = 30
if edad >= 30:
    print("Eres mayor de edad")
else:
    print("Eres menos de edad")
```

- Diagrama  alt text

Sentencia if elif else

La sentencia elif es una abreviatura de else-if, y se utiliza cuando necesitemos verificar multiples condiciones, una tras otra. Se puede crear cuantas elif necesitemos.

```
# Sintaxis sentencia else-if
if condition1:
    #Bloque de codigo
elif condition2:
    #Bloque de codigo
else:
    #Bloque de codigo
#Ejemplo
edad = 30
if edad >= 18:
    print("Eres mayor de edad")
elif 13 <= edad < 18:
    print('Eres un adolescente')
else:
    print("Eres un niño")
```

Operador ternario

Es una funcion compacta de agregar una condicion y el objetivo es asignar un valor a una variable dependiendo del valor de la condicion.

```
# Sintaxis operador ternario
resultado = valor_si_verdadero if condition else valor_si_falso
# Ejemplo
edad = 18
es_adulto = 'Si' if edad >= 18 else 'No'
```

```
print(es_adulto)
```

Seccion 8

Ciclos

Son estructuras de control que va a repetir una serie de instrucciones hasta que se cumpla una condicion especifica. Solo hay dos estrcuturas de ciclos For y While.

Ciclo While

Repite una serie de intrucciones mientras la condicion sea verdadera.

```
# Sintaxis de bucle while
while condicion:
    # Bloque de codigo a ejecutar

# Ejemplo de imprimir del 1 al 3
contador = 1
while contador <= 3:
    print(contador)
    contador +=1
```

Ciclo For

En el ciclo for itera o recorre una secuencia de valores, por ejemplo los caracteres de una cadena, una lista, etc. Y ejecuta un bloque de codigo por cada elemento de la secuencia.

```
# Sintaxis de bucle while
for variable in secuencia:
    # Bloque de codigo a ejecutar

# Ejemplo de imprimir del 1 al 3
secuencia = "Hola Mundo"
for letra, in cadena:
    print(letra, end = " ")
```